Next.js enforces specific conventions for defining route handlers, offering a structured and flexible approach to backend logic. Here are the conventions used in the previous example:

- `export async function POST(req)`—This syntax defines an asynchronous function that handles a specific `POST HTTP` method. It allows us to define different HTTP methods such as `GET`, `PATCH`, and `HEAD`.
- `export const dynamic = 'force-dynamic'`—This line is optional and configures caching behavior for the route handler. By exporting a variable with the specific name "dynamic," Next.js understands how to configure certain caching strategies. By default, dynamic is set to `auto`, enabling caching with the `GET` method and `Response` object. Setting it to `force-dynamic` means that requests are always reevaluated on the server, preventing outdated responses for frequently changing data, such as the OpenAI chat interaction.
- `req.json` and `Response.json`—These helpers are provided by Next.js to parse or respond with JSON as part of the request/response cycle.

Using Next.js's conventional approach for our backend has provided significant benefits for code maintainability and functionality. Arguably, configuring caching behavior for dynamic routes allows us to optimize performance based on the specific needs of each API endpoint. While some of Next.js's conventions are immediately intuitive, others might require more practice to understand.

2.8.7 Going deeper with Next.js

You now have all you need to continue improving your app. For readers who want to go beyond these basic approaches and build highly optimized, scalable apps, this section explores advanced Next.js features, including rendering strategies, data-fetching techniques, and performance optimization.

Advanced data fetching and rendering

Next.js offers a powerful and flexible rendering model. Understanding when and how to use each strategy is key to building a high-performance application:

- *Server-side rendering*—This is a feature where the page's HTML is generated on the server for each request. This is ideal for pages with dynamic, frequently changing data that needs to be up to date for every user. You can implement this by adding the line `export const dynamic = 'force-dynamic'` in the app router, which ensures that the route handler is reevaluated on every request.
- *Static site generation*—This is a prerendering strategy where the HTML is generated at build time. The static HTML files can be served directly from a CDN, making them incredibly fast to load. This is best for pages with static content, such as a blog post or a product page that doesn't change often.
- *Incremental static regeneration*—This is a hybrid approach that allows you to update static pages after you've built your site. You can use revalidation in a fetch request